In [1]:
```
#python datat types(integer,float,boolean)
```

In [2]:
```
i=5
type()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[2], line 2
      1 i=5
----> 2 type()

TypeError: type() takes 1 or 3 arguments
```

In [ ]:
```
type(i)
```

In [3]:
```
id(i)
```

Out[3]:
```
140718365975592
```

In [4]:
```
f=32.4
f
```

Out[4]:
```
32.4
```

In [5]:
```
type(f)
```

Out[5]:
```
float
```

In [6]:
```
f1=26e3
f1
```

Out[6]:
```
26000.0
```

In [7]:
```
f2=33.23e2
f2
```

Out[7]:
```
3323.0
```

In [8]:
```
f3=26.35e0
f3
```

Out[8]:
```
26.35
```

In [9]:
```
#boolean
```

In [10]:
```
true=6
true
```

Out[10]:
```
6
```

In [11]:
```
true
```

Out[11]: 6

In [12]:
```
True
```

Out[12]: True

True+False

In [13]:
```
#Because True =1 and False =0
```

In [14]:
```
False//True
```

Out[14]: 0

In [15]:
```
False/True
```

Out[15]: 0.0

In [16]:
```
#"/"--it gives gives decimal value like 10.3(called true div used in scientific cal
#"//"--it gives whole number rounding it to 10
```

In [17]:
```
True/False
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
Cell In[17], line 1
----> 1 True/False

ZeroDivisionError: division by zero
```

complex data type exclusively only in python

In [18]:
```
#complex is used in complex calculations where 10 is real number and 20j is imagina
#J is Square root of (-1) and J always comes at end of the
```

In [19]:
```
c=10+20j
c
```

Out[19]: (10+20j)

In [20]:
```
c.real
```

Out[20]: 10.0

In [21]:
```
#we call most of functions with  .tab i.e,c.imag or c.real
```

In [22]:
```
c.imag
```

Out[22]: 20.0

In [23]:
```
c1=3+4j
c1
```

Out[23]:
```
(3+4j)
```

In [24]:
```
print(c)
print(c1)
```

```
(10+20j)
(3+4j)
```

In [25]:
```
print(c+c1)
```

```
(13+24j)
```

In [26]:
```
c+c1
```

Out[26]:
```
(13+24j)
```

string has to be declared with single quotes or double quotes or triple quotes

In [27]:
```
'''moulya shivakoti'''
```

Out[27]:
```
'moulya shivakoti'
```

In [28]:
```
#single & double line quotes are used for single statement comments
# triple quotes only allows multiple lines of quotes
```

In [29]:
```
'moulya
shivakoti'
```

```
  Cell In[29], line 1
    'moulya
    ^
SyntaxError: unterminated string literal (detected at line 1)
```

In [30]:
```
''moulya
shivakoti''
```

```
  Cell In[30], line 1
    ''moulya
      ^
SyntaxError: invalid syntax
```

In [31]:
```
'''moulya
shivakoti'''
```

Out[31]:
```
'moulya\nshivakoti'
```

In [32]:
```
#string indexing has forward and backward indexing
# fwd starts with 0 and Bwd starts with -1
```

```
# >>> for fwd indexing and <<<< for bwd indexing
```

In [33]:
```
s1='moulya'
s1[2]
```

Out[33]:
```
'u'
```

In [34]:
```
s1[-3]
```

Out[34]:
```
'l'
```

In [35]:
```
s1[8]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[35], line 1
----> 1 s1[8]

IndexError: string index out of range
```

In [36]:
```
#Slicing is a concept where specific characters can be executed from a string
```

In [37]:
```
s2='welcome'
s2[:] #it gives all characters of a string
```

Out[37]:
```
'welcome'
```

In [38]:
```
s2[3:] # prints from 3rd index to all remaining chars of string
```

Out[38]:
```
'come'
```

In [39]:
```
s2[:5] # print from start to 4th index
```

Out[39]:
```
'welco'
```

In [40]:
```
s2[3:7] #printing value from 3rd to 6th index
```

Out[40]:
```
'come'
```

In [41]:
```
s2
```

Out[41]:
```
'welcome'
```

In [42]:
```
s2[::2]
```

Out[42]:
```
'wloe'
```

In [44]:
```
s2[::3]
```

Out[44]:
```
'wce'
```

In [45]:
```
s2[1:2:5]
```

Out[45]: 'e'

In [46]:
```
s2[::-1]
```

Out[46]: 'emoclew'

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: